Assignment 2: Convolutional Neural Networks

Chien Hsing Hu ch4022

Part 1: CNN architecture

The model is SimpleCNN in app/cnn/model.py. Input is a 3 × 64 × 64 image (CIFAR10 images resized to 64 × 64).

Layer	Output shape
Conv2d(3→16, 3×3, stride 1, padding 1) + ReLU	16 × 64 × 64
MaxPool2d(2, 2)	16 × 32 × 32
Conv2d(16→32, 3×3, stride 1, padding 1) + ReLU	32 × 32 × 32
MaxPool2d(2, 2)	32 × 16 × 16
Flatten	8192
Linear(8192→100) + ReLU	100
Linear(100→10)	10 (CIFAR10 classes)

Training uses Adam (lr 1e-3), cross-entropy loss, batch size 64 and 5 epochs. The final test accuracy on CIFAR10 is about 66%. Training and the API share the same preprocessing: Resize(64, 64), ToTensor, Normalize(0.5, 0.5).

Part 2: API, Docker and GitHub

GitHub repository: https://github.com/calebhu1110/sps-genai-ch4022-assignment1 (branch assignment-2-cnn, merged into main).

The trained weights (app/cnn/model.pth) are loaded once at start-up in eval mode. The router in app/cnn/api.py is included in the existing FastAPI app from Assignment 1, so the earlier /embedding endpoint still works.

- POST /predict_cnn: upload an image file; the response is JSON with predicted_class, class_index and confidence (softmax probability). An invalid image returns HTTP 400.

Run with Docker:

```
docker build -t sps-genai .
docker run -p 8000:80 sps-genai
```

Then open http://localhost:8000/docs and try POST /predict_cnn in Swagger UI. To retrain: uv run --group train python -m training.train.

Part 3: Theory questions

Q1. Input 32×32×3, 8 filters of 5×5, stride 1, no padding. Output size?

(32 − 5 + 2·0) / 1 + 1 = 27 + 1 = 28. There are 8 filters, so the output is **28 × 28 × 8**.

Q2. What if the padding is "same"?

For stride 1, "same" padding means P = (F − 1) / 2 = (5 − 1) / 2 = 2. Then (32 − 5 + 2·2) / 1 + 1 = 31 + 1 = 32. The output is **32 × 32 × 8**: the spatial size is preserved (it no longer shrinks from 32 to 28), and the depth is still 8.

Q3. 3×3 filter, stride 2, no padding, 64×64 input. Output spatial size?

⌊(64 − 3 + 0) / 2⌋ + 1 = ⌊61 / 2⌋ + 1 = ⌊30.5⌋ + 1 = 30 + 1 = 31. The output is **31 × 31** (the last row and column do not fit a full window, so they are dropped by the floor).

Q4. 2×2 max pooling, stride 2, on a 16×16 feature map. Output size?

(16 − 2) / 2 + 1 = 7 + 1 = 8. The output is **8 × 8** per channel; max pooling shrinks each map but keeps the number of channels unchanged.

Q5. A 128×128 image goes through two 3×3, stride 1, "same"-padding convolutions. Output shape?

Same padding for a 3×3 filter is P = 1. Each layer: (128 − 3 + 2·1) / 1 + 1 = 128. After the first layer the map is 128 × 128 × C_1, after the second 128 × 128 × C_2. The output shape is **128 × 128 × C_2**, where C_2 is the number of filters in the second convolution (the spatial size never changes).

Q6. What happens if model.train() **is removed before training?**

model.train() puts the model in training mode. This only matters for layers whose behaviour differs between training and evaluation: Dropout (randomly zeroes units in training, does nothing in eval) and BatchNorm (uses batch statistics and updates running averages in training, uses the stored running statistics in eval). A PyTorch module is in training mode by default, so on a freshly created model, removing the call changes nothing. The problem appears if model.eval() was called earlier (for example, evaluating, then training again, or re-running the training cell in a notebook): without model.train() the model stays in eval mode, so Dropout is disabled and BatchNorm does not update its statistics, which can hurt training and generalization. Gradients are still computed in eval mode (only torch.no_grad() turns them off), so training still runs but behaves incorrectly.

In the in-class notebook, train_model calls model.train() before the epoch loop. That model has no Dropout or BatchNorm layers, so the numerical result would be identical here, but the call is still good practice and is required as soon as such layers are added or .eval() has been called before.